

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут» імені Ігоря Сікорського
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота №3

Тема: Основи проектування розгортання

Виконав:
студент групи ІА-34
Єресько І.С
Перевірив:
Мягкий М.Ю

Київ-2025

Короткі теоретичні відомості

1. Діаграма розгортання (Deployment Diagram)

Діаграма розгортання є UML-діаграмою, яка описує фізичну структуру програмної системи та середовище, у якому вона функціонує. Вона показує, на яких саме фізичних або віртуальних пристроях розміщені частини програмного забезпечення, які артефакти на цих пристроях виконуються, а також яким чином ці пристрої з'єднані між собою. У такій діаграмі система представлена у вигляді вузлів: це можуть бути сервери, клієнтські комп'ютери, мобільні телефони, контролери або віртуальні сервери, розміщені в хмарному середовищі. Кожен вузол містить певні програмні елементи, наприклад веб-застосунки, служби, бази даних або файли конфігурації. Зв'язки між вузлами відображаються як мережеві канали комунікації, що дозволяють системі функціонувати як єдине ціле.

Основне призначення діаграми розгортання полягає у відображенні того, де і в якому середовищі виконується система, а також у фіксації мережевої інфраструктури, яка забезпечує її роботу. У типовій архітектурі веб-застосунку така діаграма може містити клієнтський пристрій користувача, який звертається до веб-сервера через браузер, сам веб-сервер, що відповідає за роботу інтерфейсу та прийом запитів, сервер застосунків, який виконує бізнес-логіку, і сервер бази даних, на якому зберігається інформація. Таким чином, діаграма розгортання забезпечує наочне уявлення про фізичну інфраструктуру системи і дозволяє оцінити її продуктивність, масштабованість, надійність і можливі точки відмови.

2. Діаграма компонентів (Component Diagram)

Діаграма компонентів є UML-діаграмою, що використовується для представлення логічної структури програмної системи. Вона демонструє, з яких модулів складається система, які функції виконують ці модулі та яким чином вони взаємодіють один з одним. Компоненти у цій діаграмі розглядаються як автономні частини програмного забезпечення, що мають власну відповідальність та чітко визначені

інтерфейси, через які вони надають свої функції іншим компонентам. Завдяки цьому діаграма дозволяє побачити архітектуру системи без заглиблення у програмний код.

Кожен компонент представляє певну частину функціональності. Це може бути модуль користувацького інтерфейсу, який відповідає за взаємодію з користувачем, модуль авторизації, який перевіряє правильність введених облікових даних, модуль обробки бізнес-логіки, який здійснює основні операції, або модуль доступу до бази даних, який реалізує роботу зі сховищем. Компоненти можуть викликати функції один одного через інтерфейси, а їх взаємозв'язки відображають залежності між частинами системи. Наприклад, компонент інтерфейсу залежить від компонента API, API залежить від сервісів, а сервіси - від репозиторіїв і бази даних.

Завдяки діаграмі компонентів можна зрозуміти, як саме поділена система на логічні частини, які з цих частин є ключовими, як саме вони взаємодіють між собою та які мають обов'язки. Це робить діаграму важливим інструментом для проєктування архітектури, комунікації між розробниками, а також для визначення можливостей розширення або модернізації системи.

3. Діаграма послідовностей (Sequence Diagram)

Діаграма послідовностей є UML-діаграмою, яка описує динамічну поведінку системи, тобто порядок виконання дій у межах певного сценарію. На відміну від діаграми компонентів, що показує статичну структуру системи, діаграма послідовностей демонструє, як об'єкти або актори взаємодіють між собою у часі. Вона відображає учасників процесу у вигляді вертикальних ліній, які називаються життєвими лініями, а передавання повідомлень між ними - у вигляді горизонтальних стрілок.

Діаграма показує точний порядок викликів і відповідей під час виконання певної функції. Наприклад, у сценарії авторизації користувач вводить логін і пароль, після чого інтерфейс надсилає запит на сервер контролеру авторизації. Контролер передає отримані дані у сервіс

авторизації, який перевіряє їх за допомогою репозиторію, що звертається безпосередньо до бази даних. Коли перевірка завершується, результат повертається назад до сервісу, потім до контролера і врешті - до інтерфейсу, який відображає повідомлення про успішний або неуспішний вхід.

Діаграма послідовностей дозволяє чітко зрозуміти, як саме проходить обмін даними, які компоненти беруть участь у виконанні операції, у якому порядку викликаються методи та які саме повідомлення передаються між об'єктами. Вона є незамінною для аналізу складних алгоритмів, пошуку помилок у логіці, оптимізації взаємодій між компонентами та документування функціональних сценаріїв системи.

Тема: Основи проектування розгортання.

Мета: Навчитися проектувати діаграми розгортання та компонентів для системи що проектується, а також розробляти діаграми взаємодії, а саме діаграми послідовностей, на основі сценаріїв зроблених в попередній лабораторній роботі

Хід роботи

8. Powershell terminal (strategy, command, abstract factory, bridge, interpreter, client-server)

Термінал для powershell повинен нагадувати типовий термінал з можливістю налаштування кольорів синтаксичних конструкцій, розміру вікна, фону вікна, а також виконання команд powershell і виконуваних файлів, а також працювати в декількох вікнах терміналу (у вкладках або

одночасно шляхом розділення вікна)



Рис 1: Діаграми варіантів використання.

Deployment diagram

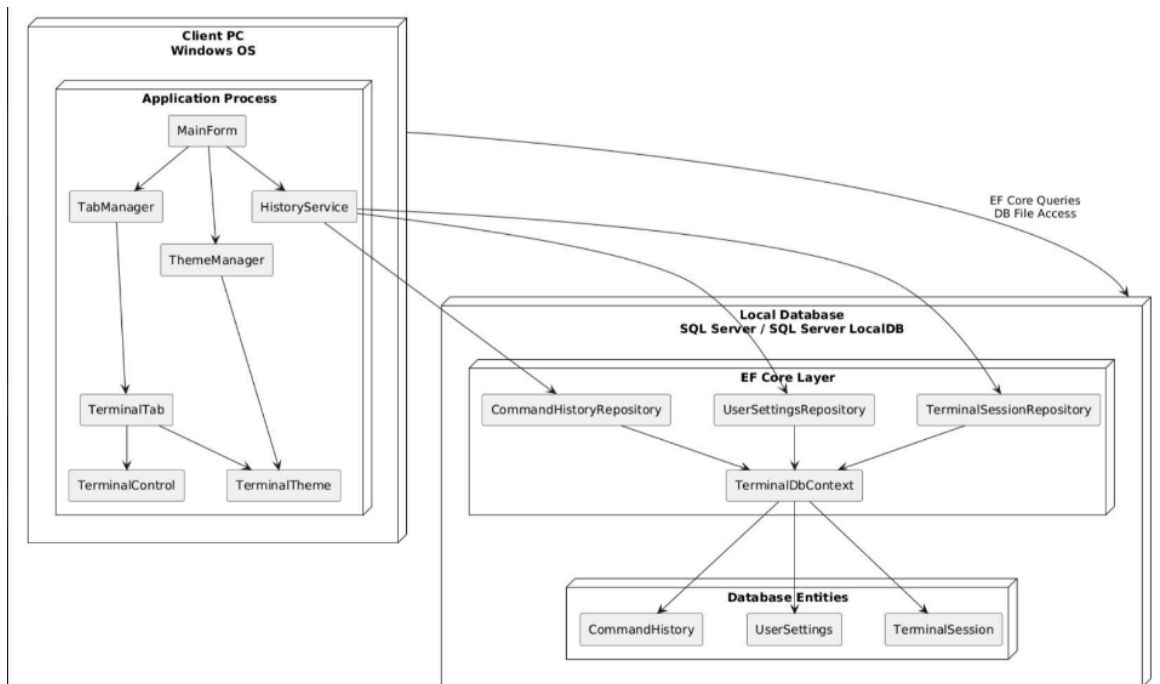


Рис 2: Діаграма розгортання

Client PC (Windows)

Містить розгорнуту WinForms/WPF/MAUI програму: MainForm, TabManager, ThemeManager, HistoryService, TerminalTab, TerminalControl, TerminalTheme.

Local Database (SQLite)

Містить: EF Core рівень (DbContext + Repositories), Таблиці (CommandHistory, UserSettings, TerminalSession).

Зв'язок

EF Core здійснює запити з застосунку до локальної БД.

Components diagram

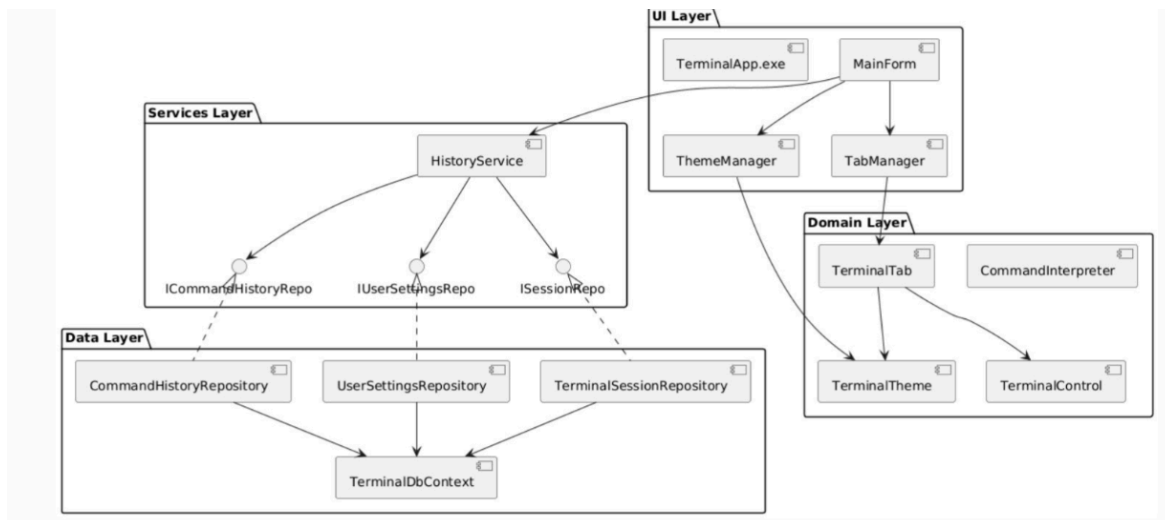


Рис 3 :Діаграма Компонентів

Діаграма компонентів показує архітектуру застосунку, поділену на чотири рівні: **UI**, **Domain**, **Services** та **Data**. Кожен рівень має свої компоненти й чіткі залежності між ними, що відображає розділення відповідальностей.

UI Layer містить виконуваний файл `TerminalApp.exe` і головну форму `MainForm`, яка керує інтерфейсом, взаємодіє з `TabManager`, `ThemeManager` та звертається до `HistoryService` для роботи з історією команд і налаштуваннями.

Domain Layer включає основні сутності: `TerminalTab`, `TerminalControl`, `TerminalTheme` та `CommandInterpreter`. Вони описують логіку роботи вкладок, команд і тем оформлення. UI працює з доменними об'єктами через `MainForm` і `TabManager`.

Services Layer містить `HistoryService`, що виступає посередником між інтерфейсом/доменом і даними. Сервіс залежить від абстракцій `ICommandsHistoryRepo`, `IUserSettingsRepo` і `ISessionRepo`, що відповідає принципу інверсії залежностей.

Data Layer реалізує ці інтерфейси через `CommandHistoryRepository`, `UserSettingsRepository` і `TerminalSessionRepository`. Усі вони використовують спільний `TerminalDbContext` для доступу до бази даних.

Sequence diagram

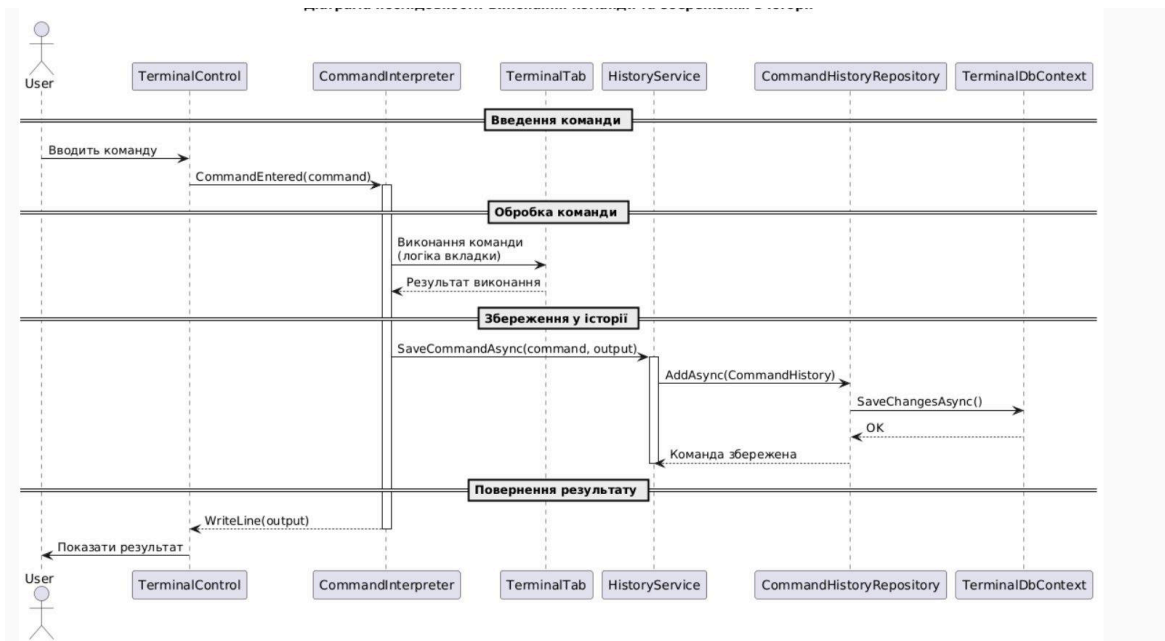


Рис 4: діаграма послідовності для варіанту використання "виконання команд"

1. Користувач вводить команду у терміналі.
Користувач вводить текст у TerminalControl.
2. TerminalControl викликає подію CommandEntered(command).
Команда передається до CommandInterpreter для подальшої обробки.
3. CommandInterpreter запускає обробку команди.
Він передає команду до TerminalTab, де виконується бізнес-логіка конкретної вкладки.
4. TerminalTab виконує логіку команди.
Тут обчислюється результат:
"Виконання команди (логіка вкладки)" - повертається результат виконання.
5. Після виконання команда передається на збереження в історію.
CommandInterpreter викликає у HistoryService метод:
SaveCommandAsync(command, output).
6. HistoryService формує об'єкт CommandHistory та передає його в CommandHistoryRepository.

Викликається:

AddAsync(CommandHistory).

7. CommandHistoryRepository викликає збереження в БД через DbContext.

Викликається:

SaveChangesAsync() - ОК.

База відповідає, що запис збережено.

8. HistoryService повертає підтвердження, що команда збережена.

9. CommandInterpreter повертає результат виконання назад у TerminalControl.

Результат виводиться на екран.

10. TerminalControl показує результат користувачу (WriteLine(output)).

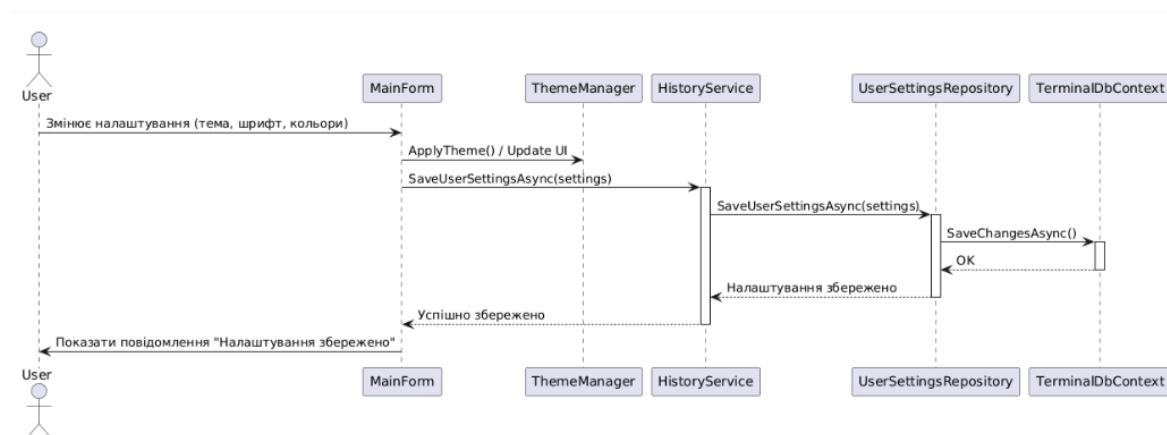


Рис 5: діаграма послідовності для варіанту використання "налаштування інтерфейсу"

1. Користувач змінює налаштування інтерфейсу

Подія відправляється у MainForm.

2. MainForm застосовує зміни до інтерфейсу через ThemeManager.

Викликається:

ApplyTheme(), ChangeFontSize(), ChangeTextColor() тощо.

3. MainForm запускає процес збереження налаштувань.
Викликається метод:
`HistoryService.SaveUserSettingsAsync(settings)`
де `settings` - нові параметри інтерфейсу.
4. HistoryService передає UserSettingsRepository дані для збереження.
Метод:
`UserSettingsRepository.SaveUserSettingsAsync(settings)`
5. UserSettingsRepository викликає TerminalDbContext.
Відбувається запис налаштувань у БД через:
`SaveChangesAsync()`
6. База даних підтверджує успішне збереження.
OK повертається в UserSettingsRepository.
7. UserSettingsRepository підтверджує збереження HistoryService.
8. HistoryService повертає відповідь у MainForm.
Повідомлення: "Налаштування збережені".
9. MainForm показує користувачу результат.
На екрані з'являється повідомлення:
«Налаштування інтерфейсу успішно збережено».

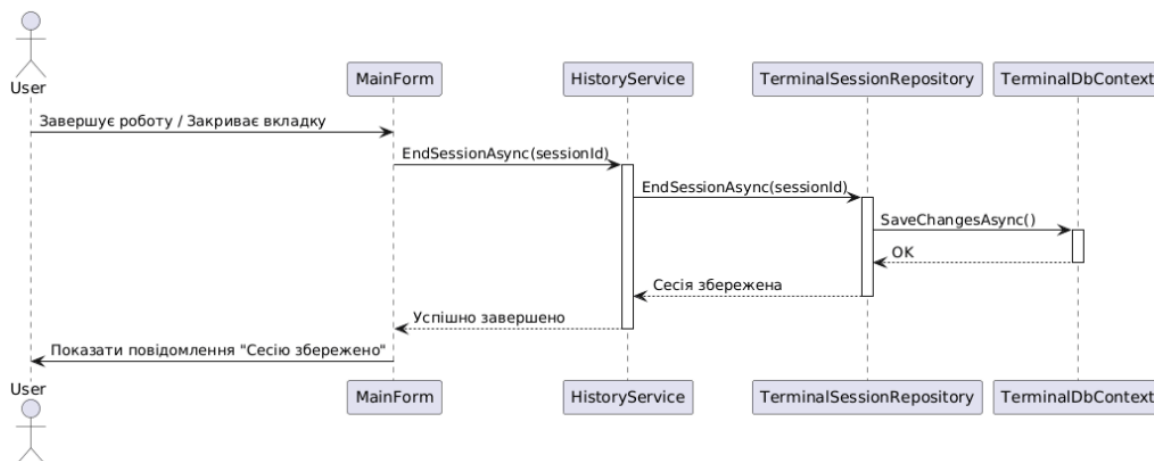


Рис 6: діаграма послідовності для варіанту використання "Збереження історії терміналів"

1. Користувач завершує роботу з терміналом.
Подія надходить у MainForm.
2. MainForm викликає завершення сесії в HistoryService.
Метод:
EndSessionAsync(sessionId)
де sessionId - активна сесія терміналу.
3. HistoryService передає команду до TerminalSessionRepository.
TerminalSessionRepository.EndSessionAsync(sessionId)
Змінюється статус сесії:
 - IsActive = false
 - EndedAt = DateTime.Now
4. TerminalSessionRepository викликає TerminalDbContext.
Виконується запис змін у БД:
SaveChangesAsync()
5. База даних повертає OK.
Інформація про завершення сесії збережена.

6. TerminalSessionRepository повертає відповідь HistoryService.

«Сесію успішно завершено.»

7. HistoryService повертає відповідь MainForm.

Сервіс повідомляє, що завершення сесії пройшло успішно.

8. MainForm показує результат користувачу.

Виводиться повідомлення:

«Сесію терміналу збережено»

Висновок

У процесі виконання лабораторної роботи було створено ключові UML-діаграми, які відображають як структурну будову системи (діаграма компонентів), так і особливості її функціонування (діаграми використання та послідовностей).

Крім того, було визначено апаратне й програмне забезпечення, необхідне для роботи застосунку, що відображено на діаграмі розгортання.

Розроблені діаграми слугуватимуть основою для подальшого поглибленого вивчення архітектури системи, вдосконалення застосунку та формування практичних навичок у створенні UML-моделей.

Контрольні запитання

1. Що собою становить діаграма розгортання?

Діаграма розгортання є графічним поданням фізичної інфраструктури, на якій виконується програмна система. Вона описує, як програмні артефакти розміщуються на апаратних вузлах. Усі елементи представляють реальні сервери, клієнтські машини, мобільні пристрої або зовнішні сервіси. Діаграма відображає, де саме працюють модулі, процеси та служби, які зв'язки між пристроями встановлені та які протоколи можуть використовуватись у взаємодії. Це дозволяє чітко уявити, як логічна архітектура перетворюється на фізичне розгортання.

2. Які бувають види вузлів на діаграмі розгортання?

На діаграмі розгортання вузли можуть бути різних типів залежно від ролі в системі.

Апаратні вузли становлять собою фізичні пристрої, такі як сервери, робочі станції, маршрутизатори або мобільні телефони. Вони є основою, на якій запускаються системні компоненти.

Віртуальні вузли є логічними контейнерами, що працюють усередині апаратних вузлів: віртуальні машини, контейнери Docker або серверні середовища.

Програмні вузли можуть являти собою процеси, служби або середовища виконання (наприклад, JVM або Node.js), які приймають на себе виконання компонентів.

Усі ці види вузлів дозволяють детально відобразити, на якому рівні працюють частини системи.

3. Які бувають зв'язки на діаграмі розгортання?

Зв'язки на діаграмі розгортання показують, як вузли взаємодіють між собою.

Комунікаційні зв'язки відображають канали обміну даними, наприклад мережеві з'єднання, TCP/IP взаємодію або API-запити. Вони демонструють, які саме пристрої спілкуються та яким шляхом передається інформація.

Зв'язки розміщення показують, які програмні артефакти розгорнуті на конкретних вузлах. Це допомагає зрозуміти розподіл файлів, компонентів і сервісів у фізичній інфраструктурі.

Завдяки цим зв'язкам діаграма забезпечує цілісне уявлення про взаємодію системи у фізичному середовищі.

4. Які елементи присутні на діаграмі компонентів?

Діаграма компонентів містить кілька ключових елементів, які описують логічну структуру системи.

Компоненти є окремими самостійними частинами програмного забезпечення, що реалізують певну функціональність. Це можуть бути модулі, бібліотеки, сервіси або класи високого рівня.

Інтерфейси представляють точки взаємодії між компонентами: що вони надають або вимагають для роботи.

Артефакти є фізичними файлами — наприклад, збірками, конфігураціями, бібліотеками, які реалізують компоненти.

Пакети групують компоненти, утворюючи логічні модулі системи.

Усі ці елементи працюють разом, щоб показати архітектуру системи на рівні її логічної структури.

5. Що становлять собою зв'язки на діаграмі компонентів?

Зв'язки на діаграмі компонентів відображають взаємодію між модулями системи.

Залежності вказують, що один компонент використовує інший або потребує його функцій.

Інтерфейсні зв'язки показують, які саме інтерфейси реалізуються або викликаються компонентами.

Реалізаційні зв'язки демонструють, який артефакт втілює певний компонент.

Завдяки зв'язкам можна чітко зрозуміти, як компоненти системи обмінюються даними, які вимоги існують між ними та як реалізована внутрішня логіка.

6. Які бувають види діаграм взаємодії?

Діаграми взаємодії подають поведінку системи через обмін повідомленнями між об'єктами.

Діаграма послідовностей зосереджується на часовому порядку обміну повідомленнями, показуючи, хто й коли викликає певні операції.

Діаграма кооперації (комунікацій) акцентує увагу на структурних зв'язках між об'єктами, показуючи, як вони пов'язані й з якою послідовністю відбувається обмін повідомленнями.

Діаграма часових шкал демонструє зміни станів об'єктів у часі, показуючи вузли активності на часовій осі.

Діаграма взаємодії загального огляду поєднує можливості послідовностей і діяльності, дозволяючи описати більш складні сценарії.

7. Для чого призначена діаграма послідовностей?

Діаграма послідовностей призначена для наочного відображення динаміки роботи системи та показує, як об'єкти взаємодіють у певному сценарії. Вона демонструє послідовність повідомлень у часі, як викликаються методи, які відповіді повертаються та які об'єкти ініціюють ті чи інші дії. Це дозволяє чітко зрозуміти логіку виконання процесів і перевірити, чи коректно організована взаємодія між частинами системи.

8. Які ключові елементи можуть бути на діаграмі послідовностей?

Діаграма послідовностей містить кілька основних елементів, що формують структуру сценарію.

Учасники або об'єкти, що взаємодіють між собою, відображаються у вигляді вертикальних ліній життя.

Повідомлення показують конкретні виклики та відповіді між учасниками, формуючи основний потік взаємодії.

Лінія життя демонструє існування об'єкта у сценарії та його активність.

Області активності вказують на моменти, коли об'єкт виконує певну операцію.

Умовні блоки, цикли та альтернативи дозволяють відобразити розгалуження логіки, циклічність дій або альтернативні сценарії.

